rogii-wellbore-geology-prediction

private 20th | public 3rd solution: RAFT-refined GR↔typewell matcher

Rank	20
Team	Yannan Chen
Author	Yannan Chen
Topic ID	733845
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733845
Retrieved with Kaggle CLI on 2026-08-10.

First, a big thank you to ROGII and the Kaggle community for this competition. Honestly one of the most fun problems I've worked on — genuinely hard, and wide open: there were so many angles worth trying. In a lot of past competitions I'd try a dozen ideas, watch them all fail, and crawl back to one simple thing. This one was the opposite. **More really is more** — a surprising number of different ideas actually worked and stacked. My main horse ended up being a deep-learning matcher; let me walk through the whole thing.

---

1. Big picture: a multi-path, decorrelated ensemble

The single most important lesson: **even when you have one strong model, uncorrelated experts still buy you a lot.** My matcher (`grtx`) alone was already gold-zone on the public LB (5.08). But it didn't stop mattering once I bolted on other paths that make their mistakes in different places.

Three paths, three mechanisms:

- `grtx` **— the main force.** A single-well GR↔typewell iterative-registration transformer. Pure single-well info: no neighbor wells, no kriging. This is where most of the score comes from.
- **Physics path —** `pfnet` **+** `pftree`**.** A GPU particle filter as a physical base, plus neural / tree models that learn the residual on top. GR + spatial prior fused the "physics" way.
- **Geometric path —** `ancc` **(kriging) +** `geotx`**.** No GR at all — pure geometry / neighbor-well geology extrapolation. Weak alone, but orthogonal to the GR matcher.

The blend is a simple per-point weighted sum, with `grtx` absorbing the residual weight:

```
out = (1 − w_pf − w_ancc − w_geotx) · grtx
      + w_ancc · ancc  +  w_geotx · geotx      ← geometric path
      + w_pf   · (pfnet / pftree)              ← physics path
```

The one non-obvious bit: the geometric path uses distance-varying weights. `w = W / (1 + (nd/D)²)`, where `nd` = distance from that point to the nearest neighbor-well ANCC control point.

- In dense well areas (`nd` small) → the geometric legs get real weight (neighbor geology is trustworthy).
- For isolated wells (`nd` large) → the weight decays to ~0 (don't trust extrapolation).
- `ancc` decays fast (D≈250–700), `geotx` decays slow (D≈1200–1500): ancc is strong up close, geotx in the mid-range, crossover around `nd≈300`.
- `pfnet` / `pftree` use **fixed** weights — per-well / per-point gating looked great on CV but died on the LB (the "selection wall", see § 6).

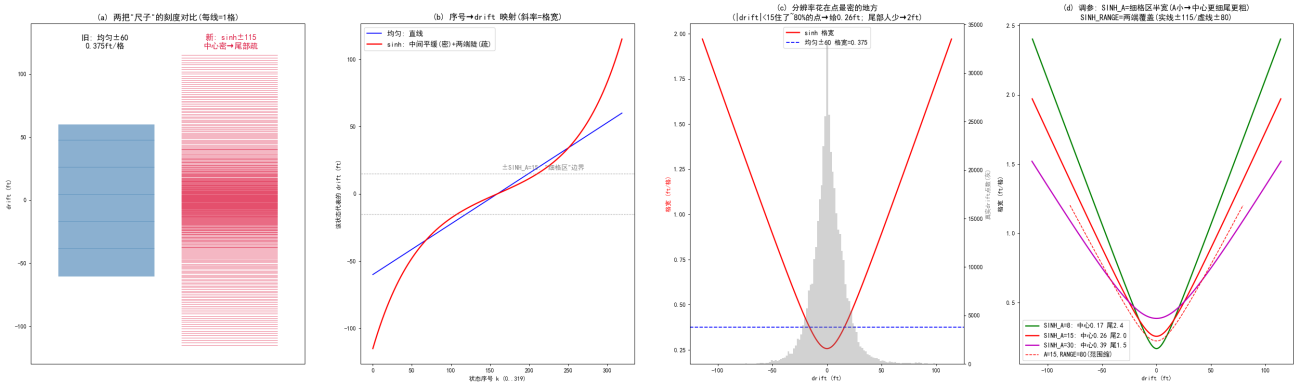
---

2. `grtx` — the main force (models referred to as `grtx-XXX`)

This is a GR-matching problem: the horizontal-well (**hw**) GR log is a distorted, shifted copy of the typewell (**tw**) GR profile, and we want the vertical `drift` (= ΔANCC − ΔZ) at every point. So `grtx` is basically a **learned image-registration model** between the two 1-D logs (hw and tw).

2.1 Data & feature handling

- **CV strategy:** plain **5-fold random** (GroupKFold by well). Every LB submission is the mean of all 5 folds.
- **Dimension choices that mattered:**
  - horizontal GR compressed to **16 ft/sample** (`COMPRESSION=16`) — 12 ft and 24 ft were both worse.
  - `N=768` samples per horizontal well; typewell resampled to `S=320` states — **everything is indexed relative to the anchor** (the known→eval transition: drift = 0 there, and the sinh state grid is centered on it).
  - `INPUT_CLIP=80`: only keep the known section within anchor±80 ft. Far-away known data is uncorrelated and actually hurts.
- **sinh non-uniform state grid (the key trick).** The drift distribution is a tight funnel — most mass is within a few ft of the anchor, but the tail reaches ±100+ ft. A uniform ±60 grid both wastes resolution in the dense center and fails to cover the tail. Instead I lay states on `g = A·sinh(b·t)`: ~0.26 ft/cell in the center (fine where the mass is), stretching out to ±115 ft in the tail (full coverage). It's analytically invertible (closed-form `asinh` to find a cell), so no binary search.



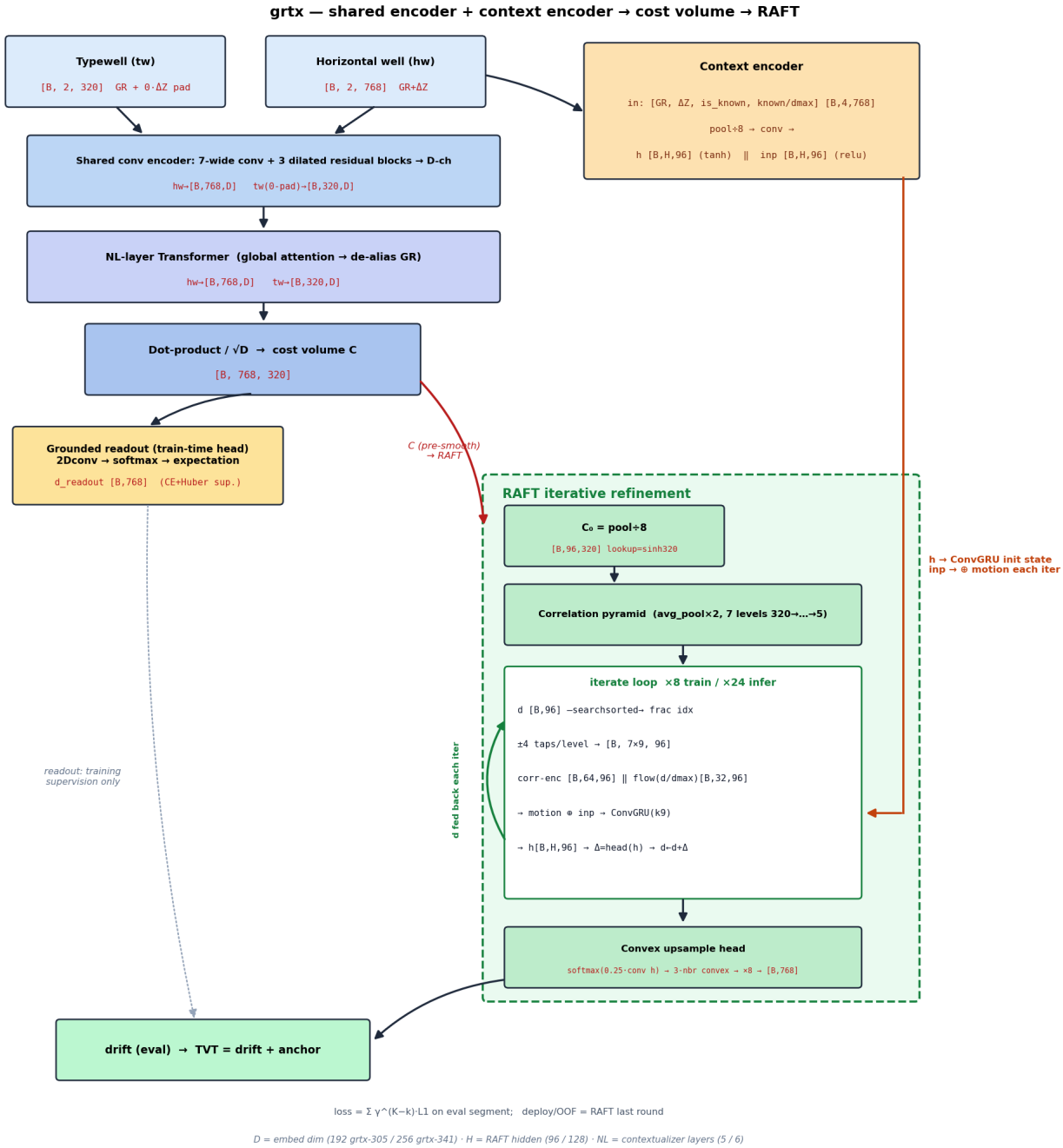
Left→right: a uniform grid vs the sinh grid; how the sinh cells map to drift; and why — cell resolution is spent exactly where the drift probability mass is.

- **Per-well joint robust norm:** hw and tw are concatenated and normalized by **one** shared median/MAD. This keeps the hw↔tw gain relationship intact while removing per-pair offset/scale (which would otherwise let the model memorize well identity).

2.2 Architecture: shared encoder → transformer → RAFT

- **Shared encoder (hw & tw).** Both logs go through the **same** conv stem (shared weights). hw carries two channels `[GR, ΔZ]`; the typewell is clean GR only, so it's given an **all-zero ΔZ channel** to match the 2-channel shape — then the identical encoder applies to both. Sharing matters: separate per-log stems made CV worse — the shared embedding is what keeps hw and tw in the same feature space so they can be matched.
- **Transformer contextualizer** (5–6 layers, dim 192–256). GR is not injective — the same little wiggle appears in many places, so local matching is ambiguous. Global attention is what disambiguates.
- **Grounded readout.** A cross-attention produces a correspondence/cost map `C`; `drift = C @ ref_drift.C` is directly supervised to peak at the true drift state (`CE_W=10`), and regularized with a 2-D conv over the cost map (stereo/optical-flow style).

- **RAFT iterative refinement.** A 1-D adaptation of **RAFT** (Teed & Deng, RAFT: Recurrent All-Pairs Field Transforms for Optical Flow, ECCV 2020 — arXiv:2003.12039): a correlation pyramid (7 levels, ±4-tap lookups each round that together cover the whole ±115 ft state axis)
  - ConvGRU updates + convex upsampling, iterating the drift estimate. Deployment uses the last RAFT iteration.



hw and tw go through a shared conv stem + transformer → cost volume; a grounded readout turns it into drift. A separate context encoder (fed by hw) produces the `h` / `inp` that drive the RAFT loop, which iteratively refines the drift.

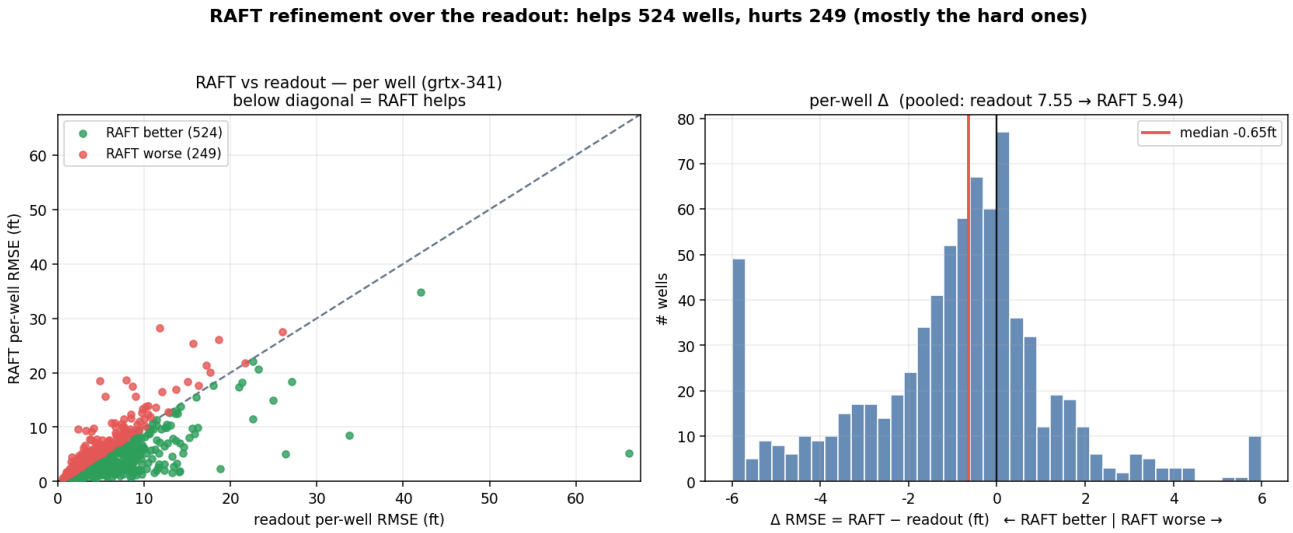
Training loss — both heads are supervised (everything on the eval segment):

- **Readout head:** a cross-entropy that peaks the cost volume `C` at the true drift state (weight 10 — the dominant term), plus a Huber on the readout drift (weight 0.2). This is what shapes a clean cost volume.

- **RAFT:** an L1 on every iteration's drift, exponentially up-weighting later rounds (`Σ γ^(K−i)·L1`, γ=0.8, weight 1.0) — the RAFT sequence loss.
- Plus a small Huber "de-damping" term that pushes the expectation toward the true drift.

Deploy / OOF uses the **RAFT last iteration**; the readout is a training-time head only.

How much does RAFT actually buy? The deployed models run **24 RAFT iterations** at inference. In tuning, ~16 was already the sweet spot — beyond that it flatlines (24 is a hair worse, well within noise) — so the exact count past 16 barely matters. Comparing the readout (pre-RAFT) against the RAFT output on the grtx-341 OOF: RAFT **improves 524 of 773 wells and hurts 249**, taking the pooled RMSE from **7.55 → 5.94**. The wells it hurts are the hard ones (the spurious-warp wells of § 6) — but the ~2.4 ft it gains on the bulk far outweighs the ~1.4 ft it gives back on the tail.



Per-well RMSE, readout vs RAFT (grtx-341). Left: points below the diagonal are wells RAFT improves (green, 524) vs wells it hurts (red, 249). Right: the per-well Δ distribution — mostly negative (RAFT better), pooled 7.55 → 5.94.

2.3 The data engine: three parallel loaders

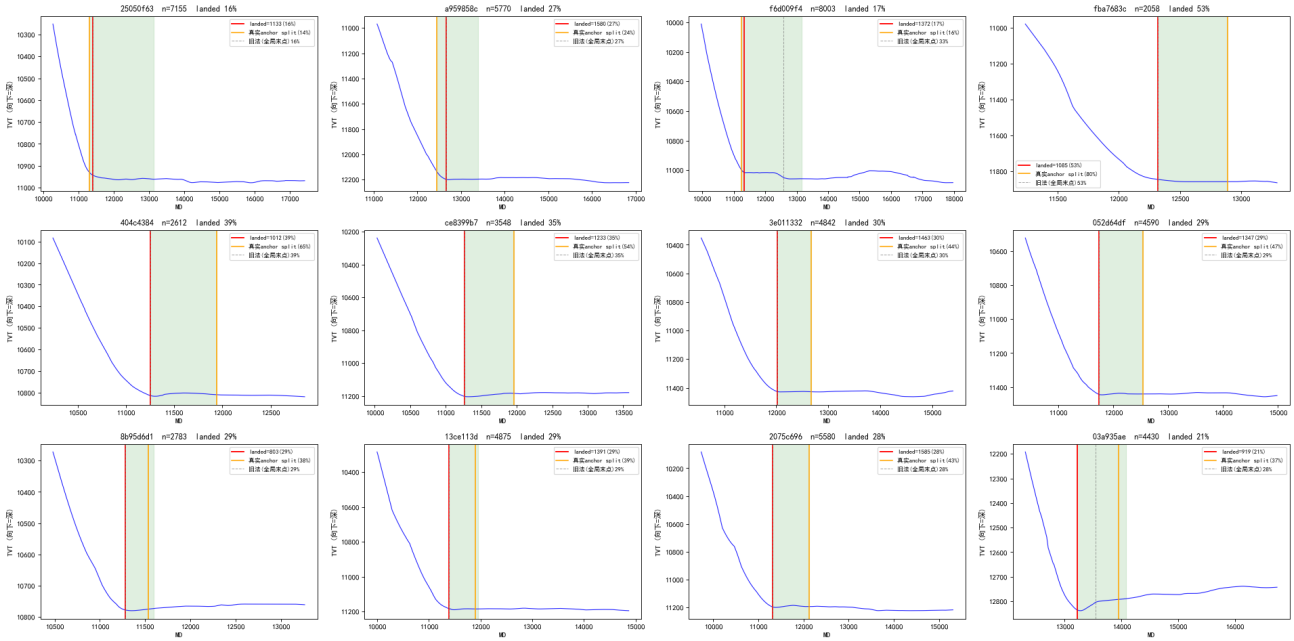
773 wells is tiny for a matcher this expressive — it overfits and plateaus. The fix is a data engine made of **three parallel loaders**, mixed live in every batch (the DataLoader workers regenerate the synthetic ones on the fly, so the model basically never sees the same well twice):

1. real wells + heavy on-the-fly augmentation,
2. **whip** — synthetic drift trajectories,
3. **layer** — bulk formation shifts.

Loaders 2 and 3 share a forward-GR simulator (the "inversion" function) that I'll describe at the end.

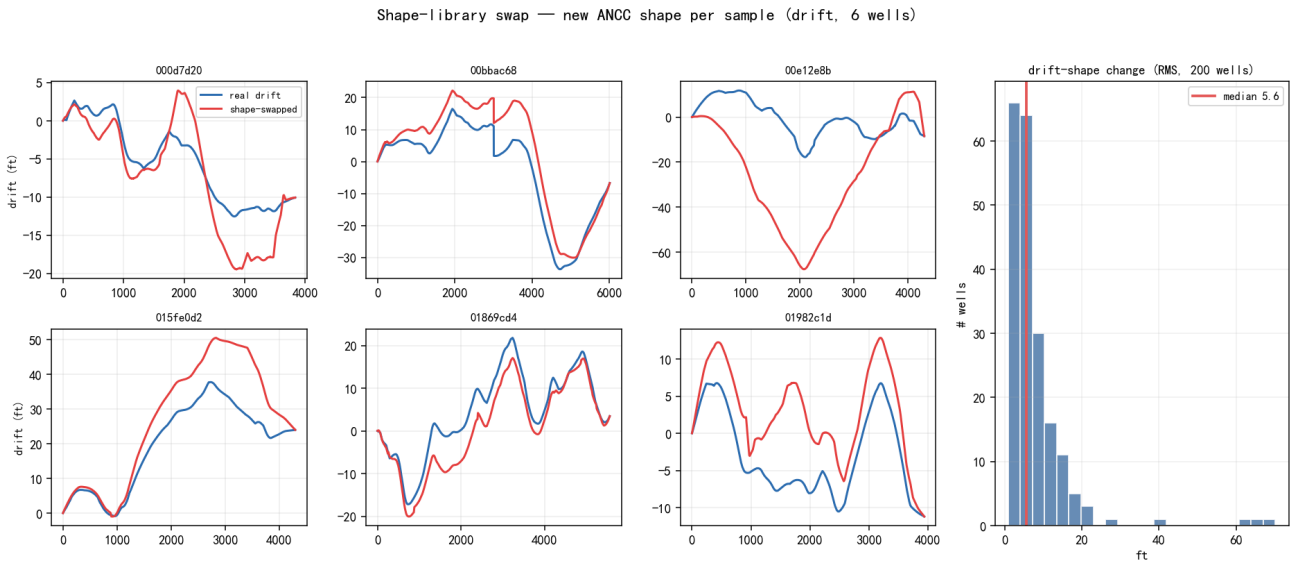
Loader 1 — real wells + augmentation

- **Landing-aware anchor slide** (this is a bigger deal than it sounds). The known/eval split isn't fixed — every epoch it's resampled anywhere from the well's landing point outward along the horizontal. So one real well becomes many training examples with different anchor positions and known-section lengths. It's the cheapest, highest-value augmentation here.



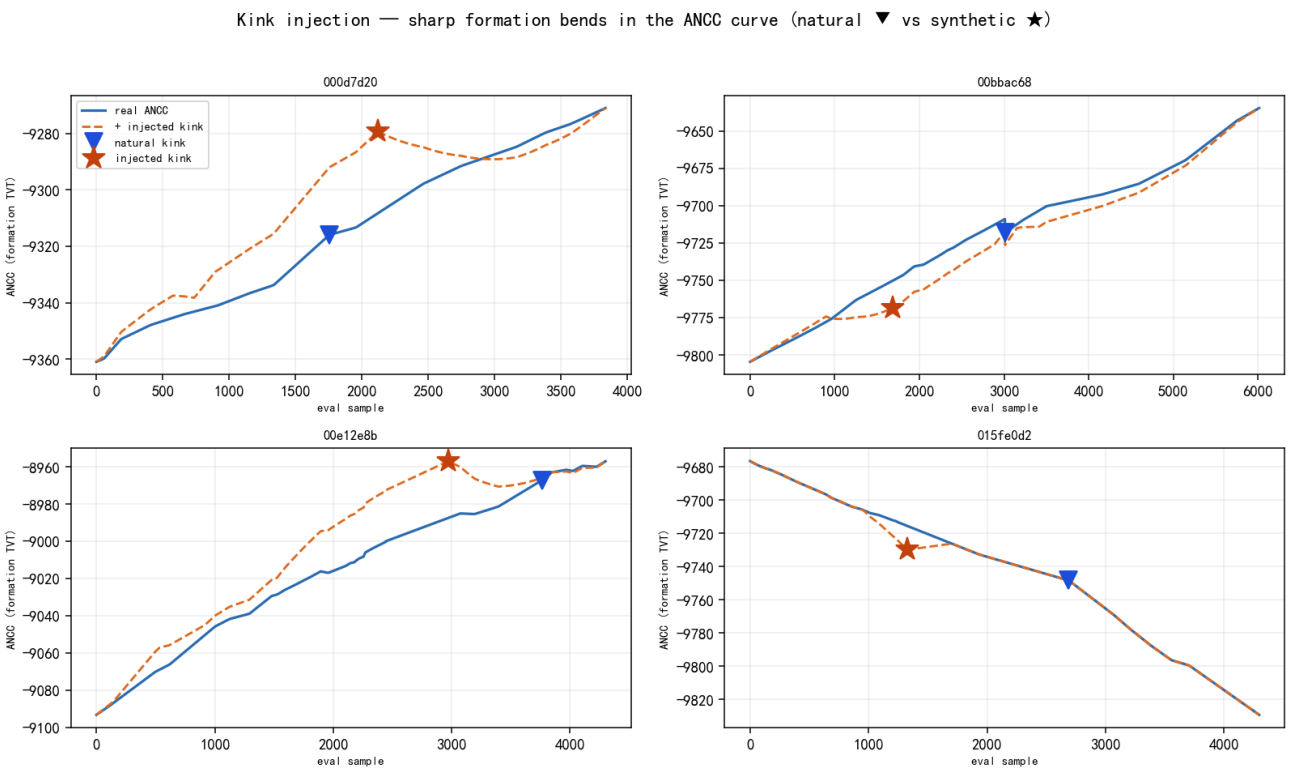
Each panel is one well's TVT vs MD. Red = landing point, orange = the "real" anchor split, green band = the range the anchor is randomly sampled from during training. The split slides freely across the landed horizontal section.

- **Typewell warp** (`WARP_P=0.8`) — deform the typewell's layer thicknesses and sync-distort the hw GR (drift/geometry unchanged): texture diversity so the matcher doesn't memorize exact log shapes.
- **ANCC shape-library swap** (`SHAPE_AUG_P=0.8`) — replace the well's ANCC shape with one drawn from a library of training-fold shapes (α-blend, variance-preserving) → a new drift shape on the same geometry.



Shape swap on 6 wells (real vs swapped drift) and the distribution of drift-shape change across wells.

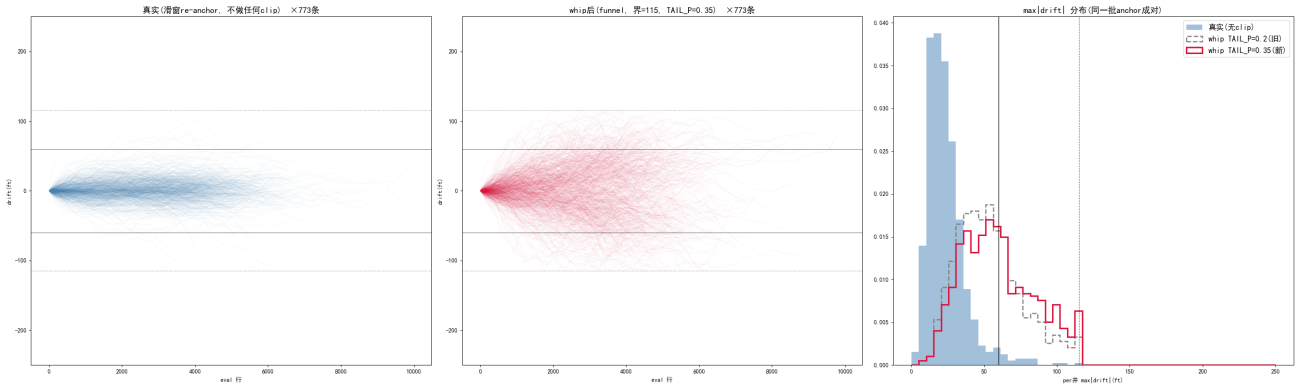
Plus optional **kink injection**: hard wells are dominated by sharp formation kinks whose natural base rate is only 2–3%, so we inject more — with a **steering response** where the wellbore ΔZ lags and follows the bend (real wells track kinks at ~0.5 ratio), so the synthetic trajectory doesn't pretend to be blind.



The ANCC (formation-depth) curve for 4 wells: blue = real (▼ marks its natural kink), orange dashed = the same well with a synthetic kink injected (★). We deliberately add these sharp bends because they're the #1 hard-well failure mode and rare in the raw data.

Loader 2 — whip (synthetic drift)

Take a real well and replace its eval-segment drift with a pinned random walk that fans out funnel-style, deliberately over-populating the tail that real wells rarely reach. Then re-simulate the GR (below).



Left: real 773-well drift (narrow funnel). Middle: whip synthetic (wider — fills the tail). Right: the max|drift| distributions, showing whip covering the region real wells under-sample.

Loader 3 — layer shift

Shift the whole well up/down → the GR samples a different formation level, while drift/ΔZ stay fixed. Orthogonal to whip (whip changes the shape of drift; layer changes which formation you're logging).

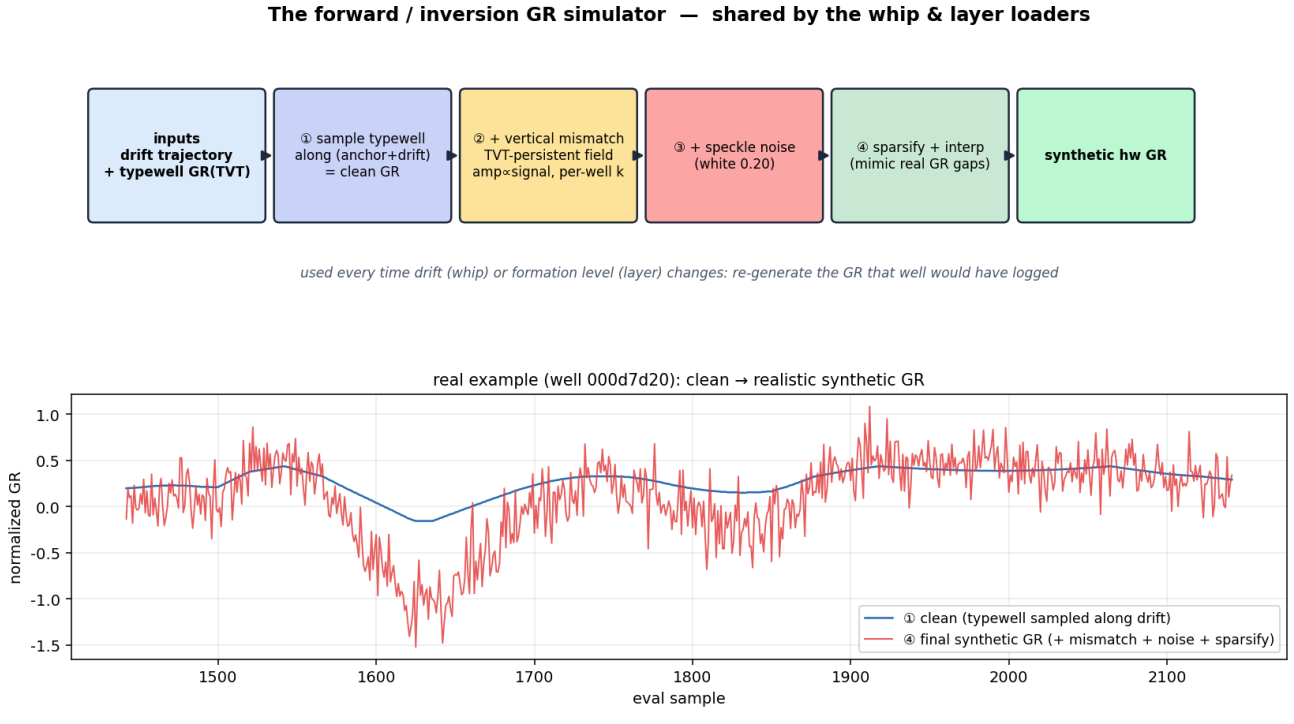


Layer shift on 6 wells (real vs shifted GR) and the applied-shift distribution.

The forward / inversion GR simulator (shared by loaders 2 & 3)

Whenever the drift (whip) or the formation level (layer) changes, the GR that the horizontal well would have logged has to be regenerated — otherwise the (GR, drift) pair is inconsistent. That's this function, and getting it realistic is what makes the synthetic wells actually help:

1. **sample** the typewell along `anchor + drift` → the clean GR (what a perfectly-tracking well would see);
2. **add vertical mismatch** — a TVT-persistent field (same depth → same distortion), amplitude scaled to the well's signal, with per-well random severity, calibrated to the real 773-well statistics (it's a vertical mismatch, not a horizontal warp — real logs line up at lag 0);
3. **add speckle noise**;
4. **sparsify + interpolate** to mimic the gaps in real GR logs.



Top: the simulator steps. Bottom: a real example — clean typewell sampling (blue) turned into a realistic synthetic GR (red) with mismatch, noise and gaps.

> Counter-intuitive lesson: **"looks more realistic" ≠ "trains better."** A visually gorgeous steep-heel synthetic variant actually made CV worse. Trust the CV, not your eyes.

---

3. Physics path: `pfnet` + `pftree` (`pfnet-XXX`, `pftree-XXX`)

3.1 GPU-ifying the particle filter (`gen_gpupf`)

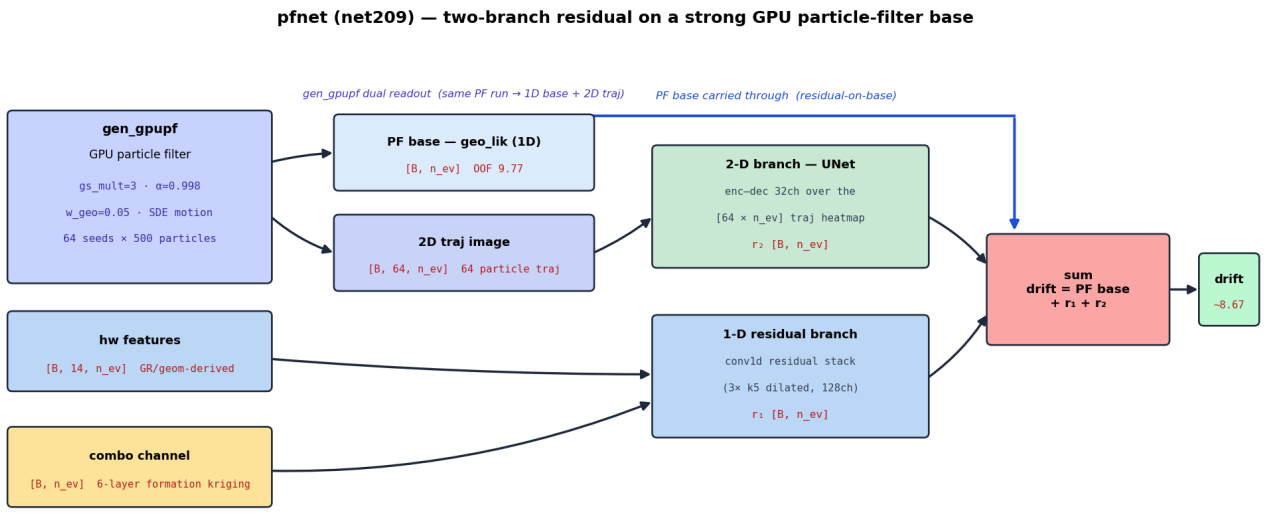
The classic geosteering tool is a particle filter over the ANCC state. The public/CPU PF is slow and weak. I rewrote it as **one GPU function** `gen_gpupf(Z, **10 params)` → `{well: drift}`, used **identically** for training-base generation and kernel inference. Ten knobs (seeds, particles, momentum α, SDE motion, emission width, kriging geo-prior weight `w_geo`, readout mode…) generate every PF "expert" from one code path. The deployed base (`gpupf_g3geo_krig`) runs with `gs_mult=3`, `α=0.998`, `w_geo=0.05`, **64 seeds × 500 particles, SDE motion** — a raw **OOF 9.77, versus ~14.35 for the competition's built-in PF.** That gap is most of the physics path's value. (`w_geo` here is the PF's internal spatial-prior weight — `gs_mult` is the emission width, `α` the dip momentum — not to be confused with the `ancc`/`geotx` blend weights in §5.)

3.2 `pfnet` architecture

A two-branch net on top of the PF base:

- a **1-D branch** learning the residual on the PF base drift,
- a **2-D branch** (UNet over the g3 particle-trajectory image),
- plus a formation-kriging "combo" channel.

The philosophy is **residual-on-a-strong-base**, which mirrors the physics: the PF base gives the big picture, the net fixes the local errors. With twin (forward-sim) augmentation on top, single-model ~8.67.



The GPU particle-filter base carries the big picture; a 1-D branch learns the residual, a 2-D UNet reads the particle-trajectory image, and a formation-kriging channel adds spatial geology — summed back onto the base.

3.3 `pftree` — a tabular residual model, and its feature study

A per-point gradient-boosting model (XGBoost + depth-diverse CatBoost → ridge stack) predicting drift from a ~195-feature bank — the "everything a single well knows about this point" set. The families, with examples:

- **Neighbor-geology kriging** — the ANCC formation surface (and a handful of related surfaces) kriged in from neighboring wells at this X/Y (leave-one-out), plus a distance/reliability feature. This is the geometric prior — where the formation should be based on the neighbors.
- **GR-matching descriptors** — match the hw GR against the typewell at several stiffnesses/windows and read off the implied drift and how much those estimates agree (disagreement = low-confidence point).
- **Particle-filter outputs** — the `gpupf` drift and its spread.
- **Local GR & geometry** — rolling GR statistics, ΔZ, heading, distance-from-anchor.

The single biggest lever was **residual-on-a-strong-base**: instead of predicting drift directly, put the `gpupf` PF drift underneath and let the trees learn only the residual (~1.5 ft single-model jump).

---

4. Geometric path

- `ancc` — spatial IDW kriging of the ANCC formation surface (K60, leave-one-out). Base drift cv ~14.6.
- `geotx` — a GR-free geometric transformer (d=160, 5 layers, 8 heads, ~1.6M params). Input = a 768-point resampled well with **9 channels** (known-drift history, is-known mask, ΔZ, dx, dy, heading cos/sin, grid-kriging neighbor geology, nn-dist reliability); it outputs drift directly. Trained with **sliding-window anchor augmentation** (the key regularizer against 773-well overfitting), 5 folds averaged. Single cv ~11.3.

Both are weak alone (11–14 ft — geology has an irreducible part that geometry simply can't see without GR). But their mechanism is **completely orthogonal to the GR matcher**, so in the blend they punch far above their single-model weight.

---

5. CV vs LB, and the submission strategy (the part that actually won gold)

5.1 Score progression (all public LB — but public LB lies at small scale, refer to § 5.2)

- single `grtx`: **5.08**
- + geometric path (`ancc`): → **5.03**
- + physics path (pfnet): → **4.90**

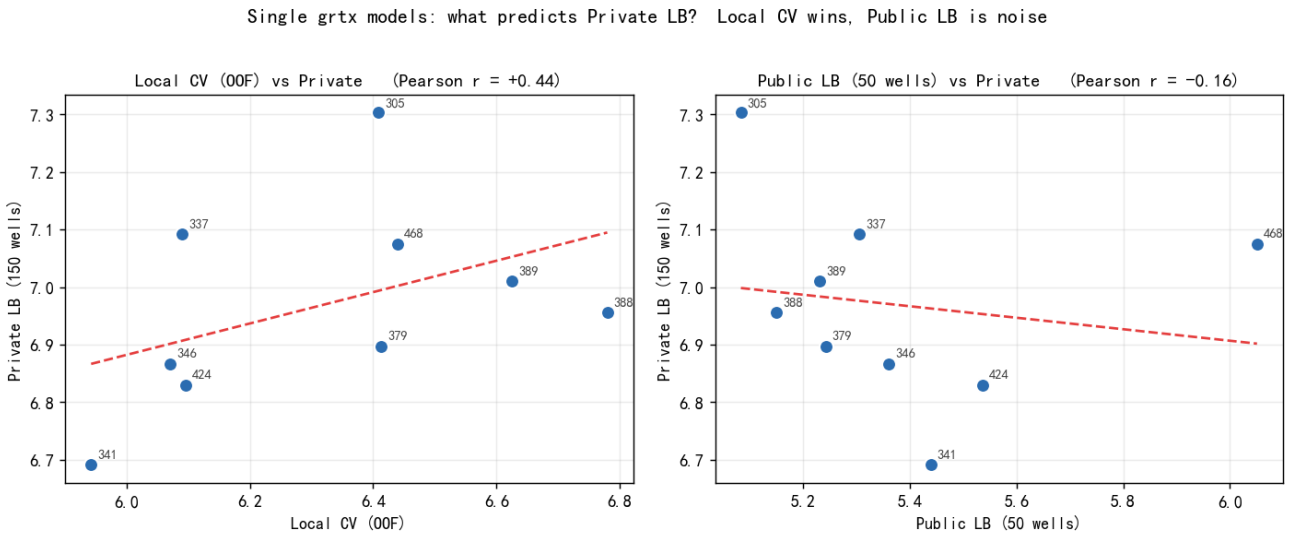
The core rule: decorrelation > single-model strength. The cleanest proof: a residual-on-`pfnet` tree had a stronger single model (8.44) but was 0.96-correlated with `pfnet` → **zero** blend gain. Meanwhile a weak GR-free geometric leg (single model only 11–14) bought −0.4 in the blend. Adding an uncorrelated leg beats strengthening an existing one, every time.

5.2 The trap: public LB is not trustworthy (and it nearly cost me gold)

Public LB here is **only 50 wells** → the noise is enormous (bootstrap σ of a 50-well score ≈ 0.9; σ of the difference between two models ≈ 0.6). That's not enough signal to pick models.

I checked this directly on my single `grtx` models:

model	CV	public LB	private LB
grtx-305	6.408	**5.083**	7.303
grtx-337	6.090	5.305	7.092
grtx-341	**5.942**	5.439	**6.691**
grtx-346	6.070	5.360	6.867
grtx-379	6.413	5.242	6.897
grtx-388	6.780	5.150	6.956
grtx-389	6.625	5.231	7.010
grtx-424	6.095	5.536	6.830
grtx-468	6.439	6.051	7.075



Local CV correlates +0.44 with private LB. Public LB correlates −0.16 — it's literally negatively correlated. Chasing public LB would have actively steered me the wrong way.

And the final blends tell the same story:

- My **highest public-LB submission I did not select** — it was an ensemble with a single grtx model with mediocre local CV.

- I selected two **larger ensembles** whose CV was better but whose public LB was worse — i.e. CV and public LB were **anti-correlated** across my candidates.
- On the final **private LB (150 wells)**, the CV ordering was the right one. The ensembles I picked held up.
- And the kicker: if I'd trusted CV even harder and picked the highest-CV / lower-public sub, I'd have scored **even better**.

strategy	CV	public LB	private LB	selected
grtx-305 + pfnet-209·0.15 + ancc·0.05@250 + geotx·0.05@1200	6.03	4.902	6.914	-
grtx-[305+379+388+389] + pfnet-209·0.15 + ancc·0.02@700 + geotx·0.04@1500	5.774	4.939	6.639	✅
grtx-[305+337+341+346+424+468] + pfnet-209·0.10 + pftree-083·0.05 + ancc·0.05@250 + geotx·0.05@1200	5.437	5.197	6.500	✅
grtx-[337+341+346+424+468] + pfnet-209·0.10 + pftree-083·0.05 + ancc·0.10@250	5.398	5.299	6.424	-

The best public-LB strategy (4.902) has the **worst** private LB (6.914). If I'd chased it, I'd have been toasted right out of the gold zone. The ones I selected — chosen on CV — are the ones that held.

Trust your local CV. Don't let an overfit public LB blind you.